

AACAP26 - OpenMP Challenge

Obiettivi e Regole

Questa challenge vi farà ragionare su vari argomenti che abbiamo trattato nel corso finora; in particolare, se torniamo ai risultati di apprendimento attesi presentati nella lezione introduttiva, allena le vostre capacità di

“sfruttare la conoscenza dell’architettura del calcolatore per migliorare le prestazioni dei programmi”.

Compilazione e consegna:

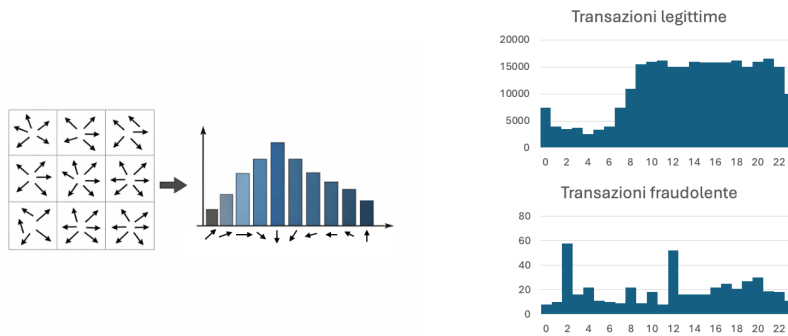
- Rispondete alle domande aggiungendo pagine in fondo a questo PDF (non cancellate la descrizione del problema).
- Salvate **due copie** del documento con le risposte, una con i nomi dei partecipanti al gruppo in una tabellina simile a quella in fondo a questa pagina e una *blind*.
- Mandate a serena.curzel@polimi.it due file:
 - Il documento PDF con i nomi;
 - Un archivio .zip contenente il documento PDF *blind* e tutto il codice ed eventuali script necessari per riprodurre gli esperimenti, sia nella versione base sia nell’eventuale versione ottimizzata.

Buon lavoro!

Nome e Cognome	Codice Persona

Descrizione del problema

Un istogramma può essere usato per identificare pattern in un dataset. Nella versione più semplice, un istogramma viene costruito analizzando il valore di ciascun elemento del dataset e incrementando il contatore corrispondente. Per esempio, la parte sinistra della figura qui sotto è un istogramma che identifica l'orientamento dei contorni rilevati in un'immagine; nella parte destra due istogrammi mostrano pattern di attività diversi nel corso di una giornata per transazioni bancarie legittime e fraudolente:



Possiamo dividere i dati di input in sezioni indipendenti, che quindi possono essere gestite in parallelo. Ecco uno snippet di codice che utilizza OpenMP per generare un istogramma che conta il numero di occorrenze di ogni lettera in una stringa di caratteri, raggruppandole in sette *bin* da quattro lettere (a-d, e-h, i-l, m-p, q-t, u-x, y-z):

```
#pragma omp parallel
{
    int tid = omp_get_thread_num();

    #pragma omp for
    for (int i = 0; i < length; ++i) {
        int alphabet_pos = (int)data[i] - 'a';
        if (alphabet_pos >= 0 && alphabet_pos < 26) {
            int bin = alphabet_pos / 4;
            local_histograms[bin][tid]++;
        }
    }
}

for (int b = 0; b < 7; ++b)
    for (int t = 0; t < total_threads; ++t)
        histogram[b] += local_histograms[b][t];
```

Domande

1. **Cosa manca per poter eseguire il programma?** Completate il codice con l'inizializzazione corretta delle variabili, e aggiungete la misurazione del tempo totale di esecuzione.
2. **Il tempo di esecuzione diminuisce all'aumentare del numero di thread utilizzati?** Eseguite dei test con diversi numeri di thread, elencate le caratteristiche della macchina che avete usato per eseguirli (sistema operativo, numero di core, versione di OpenMP), e riportate in una tabella o in un grafico i risultati.
3. **Come mai la diminuzione del tempo di esecuzione non è lineare?** Fate delle ipotesi in base alle vostre conoscenze dell'architettura del calcolatore.
4. **Si può rendere il codice più efficiente?** In base alla risposta che avete trovato per il punto 3, modificate il programma e ripetete gli esperimenti del punto 2 per dimostrare il miglioramento nel tempo di esecuzione.

AACAP26 - OpenMP Challenge

Relazione e Risposte ai Quesiti

Nome e Cognome	Codice Persona
Sofia Longo	10929711
Lorenzo Pirozzi	10932403
Maria Clotilde Raffi	10919732

4 giugno 2026

1 Inizializzazione delle variabili e misurazione del tempo

Il codice fornito ometteva la dichiarazione e l'inizializzazione delle strutture dati, oltre alla logica legata al calcolo del tempo di esecuzione. Di seguito vengono descritte le integrazioni necessarie per rendere il programma eseguibile e per condurre gli esperimenti.

1.1 Inizializzazione delle Variabili

Per consentire l'esecuzione del codice e la corretta conduzione dei test di scalabilità, le strutture dati principali sono state ridefinite sfruttando l'allocazione dinamica. Le specifiche di inizializzazione per le variabili chiave sono descritte di seguito:

- **data** e **length**: Rappresentano l'array dei caratteri di input e la sua dimensione, che funge da limite superiore (*upper bound*) per il ciclo parallelo. Per garantire la riproducibilità degli esperimenti, il buffer viene allocato dinamicamente a runtime e popolato tramite un generatore pseudocasuale basato su un *seed* fisso.
- **total_threads**: Indica il numero di thread attivi nella regione parallela. Al fine di valutare la scalabilità del sistema, questo parametro viene variato programmaticamente nel codice tramite la funzione `omp_set_num_threads()`, iterando su potenze di due. Il valore definisce inoltre il limite superiore del ciclo finale di riduzione.
- **local_histograms** e **histogram**: Sono i vettori dedicati al calcolo delle frequenze. L'array globale **histogram** viene azzerato esplicitamente per evitare la presenza di valori indefiniti in memoria. Per la matrice dei contatori locali, si è scelta un'allocazione dinamica tramite `calloc`:

```
1 int num_threads = omp_get_max_threads();
2 int (*local_histograms)[num_threads] =
3     calloc(NUM_BINS, num_threads * sizeof(int));
4 int histogram[NUM_BINS] = {0};
```

L'impiego della funzione `calloc` consente di allocare lo spazio necessario e, contemporaneamente, di azzerare i blocchi di memoria riservati. In questo modo si garantisce che i conteggi parziali di ciascun thread inizino da uno stato nullo, evitando l'introduzione di cicli di azzeramento espliciti.

1.2 Misurazione del Tempo di Esecuzione

Per valutare le prestazioni è stato utilizzato `omp_get_wtime()`, la funzione ad alta risoluzione fornita dalle API di OpenMP. Nel codice è stato inserito un *checkpoint* intermedio (`t_mid`) per isolare il tempo speso nella vera e propria regione parallela da quello richiesto dalla riduzione finale.

```
1 double t_start = omp_get_wtime();
2
3 #pragma omp parallel
4 {
5     int tid = omp_get_thread_num();
6     #pragma omp for
7     for (long i = 0; i < length; ++i) {
8         int alphabet_pos = (int)data[i] - 'a';
9         if (alphabet_pos >= 0 && alphabet_pos < 26) {
10             int bin = alphabet_pos / 4;
11             local_histograms[bin][tid]++;
12         }
13     }
14 }
15
16 // Checkpoint post-regione parallela
17 double t_mid = omp_get_wtime();
18
19 // Riduzione seriale
20 for (int b = 0; b < NUM_BINS; ++b) {
21     for (int t = 0; t < total_threads; ++t) {
22         histogram[b] += local_histograms[b][t];
23     }
24 }
25
26 double t_end = omp_get_wtime();
27
28 double tot = t_end - t_start;
29 double par = t_mid - t_start;
30 double red = t_end - t_mid;
```

Analizzando le tempistiche, è emerso subito che il costo computazionale della riduzione seriale (**red**) è inferiore di vari ordini di grandezza rispetto alla fase parallela. È essenzialmente un overhead trascurabile che non scala con la dimensione dell'array di input. Per questo motivo, nei benchmark successivi l'analisi della performance è incentrata sulla sola regione parallela (**par**).

2 Caratteristiche della macchina di test

- Architettura / Modello CPU: Intel(R) Core(TM) Ultra 7 256V
- Sistema Operativo: Ubuntu-26.04;
- Numero dei Core fisici/logici: 8;

- Thread(s) per core: 1;
- Versione di OpenMP utilizzata: 201511
- Caches:
 - L1d: 384 KiB (8 istanze);
 - L1i: 512 KiB (8 istanze);
 - L2: 20 MiB (8 istanze);
 - L3: 12 MiB (1 istanza);

Risultati dei test empirici (Versione Base - v1)

Tabella 1: Tempi di esecuzione, speedup ed efficienza estesi fino a 4096 thread (v1).

Lunghezza	Thread	T.tot (s)	T.par (s)	T.red (s)	Speedup	Efficienza
10 000 000	1	0.014255	0.014255	0.000000	1.000	1.0000
	2	0.017683	0.017682	0.000001	0.806	0.4031
	4	0.043692	0.043692	0.000001	0.326	0.0816
	8	0.056401	0.056401	0.000001	0.253	0.0316
	16	0.042233	0.042232	0.000000	0.338	0.0211
	32	0.023287	0.023287	0.000000	0.612	0.0191
	64	0.019160	0.019160	0.000000	0.744	0.0116
	128	0.025255	0.025255	0.000000	0.564	0.0044
	256	0.016446	0.016446	0.000000	0.867	0.0034
	512	0.017736	0.017736	0.000000	0.804	0.0016
	1024	0.047373	0.047373	0.000000	0.301	0.0003
	2048	0.059428	0.059428	0.000000	0.240	0.0001
	4096	0.104364	0.104364	0.000000	0.137	0.0000
50 000 000	1	0.047065	0.047065	0.000000	1.000	1.0000
	2	0.165130	0.165130	0.000000	0.285	0.1425
	4	0.384445	0.384445	0.000000	0.122	0.0306
	8	0.476626	0.476626	0.000000	0.099	0.0123
	16	0.229558	0.229558	0.000000	0.205	0.0128
	32	0.119045	0.119045	0.000000	0.395	0.0124
	64	0.068317	0.068316	0.000000	0.689	0.0108
	128	0.038246	0.038246	0.000000	1.231	0.0096
	256	0.060540	0.060540	0.000000	0.777	0.0030
	512	0.047699	0.047699	0.000000	0.987	0.0019
	1024	0.056247	0.056247	0.000000	0.837	0.0008
	2048	0.085486	0.085486	0.000000	0.551	0.0003
	4096	0.084106	0.084106	0.000000	0.560	0.0001
100 000 000	1	0.067974	0.067974	0.000000	1.000	1.0000
	2	0.378926	0.378926	0.000000	0.179	0.0897
	4	0.452371	0.452371	0.000000	0.150	0.0376
	8	1.151962	1.151962	0.000000	0.059	0.0074
	16	0.946152	0.946152	0.000000	0.072	0.0045
	32	0.422488	0.422488	0.000000	0.161	0.0050
	64	0.119128	0.119128	0.000000	0.571	0.0089
	128	0.100488	0.100487	0.000000	0.676	0.0053
	256	0.065041	0.065041	0.000000	1.045	0.0041
	512	0.077540	0.077539	0.000000	0.877	0.0017
	1024	0.070479	0.070478	0.000000	0.964	0.0009
	2048	0.094874	0.094874	0.000000	0.716	0.0003
	4096	0.101736	0.101735	0.000001	0.668	0.0002
500 000 000	1	0.233490	0.233489	0.000000	1.000	1.0000
	2	1.829699	1.829699	0.000000	0.128	0.0638
	4	2.901702	2.901702	0.000000	0.080	0.0201
	8	3.105532	3.105531	0.000001	0.075	0.0094
	16	3.821244	3.821244	0.000000	0.061	0.0038
	32	1.624393	1.624393	0.000000	0.144	0.0045
	64	0.375921	0.375921	0.000000	0.621	0.0097
	128	0.159692	0.159692	0.000000	1.462	0.0114
	256	0.327848	0.327848	0.000000	0.712	0.0028

Continua nella pagina successiva

Tabella 1 – continua dalla pagina precedente

Lunghezza	Thread	T.tot (s)	T.par (s)	T.red (s)	Speedup	Efficienza
	512	0.207262	0.207262	0.000000	1.127	0.0022
	1024	0.158066	0.158065	0.000000	1.477	0.0014
	2048	0.208734	0.208733	0.000000	1.119	0.0005
	4096	0.163677	0.163676	0.000000	1.427	0.0003

Scalabilità Baseline (v1): Speedup vs Numero di Threads

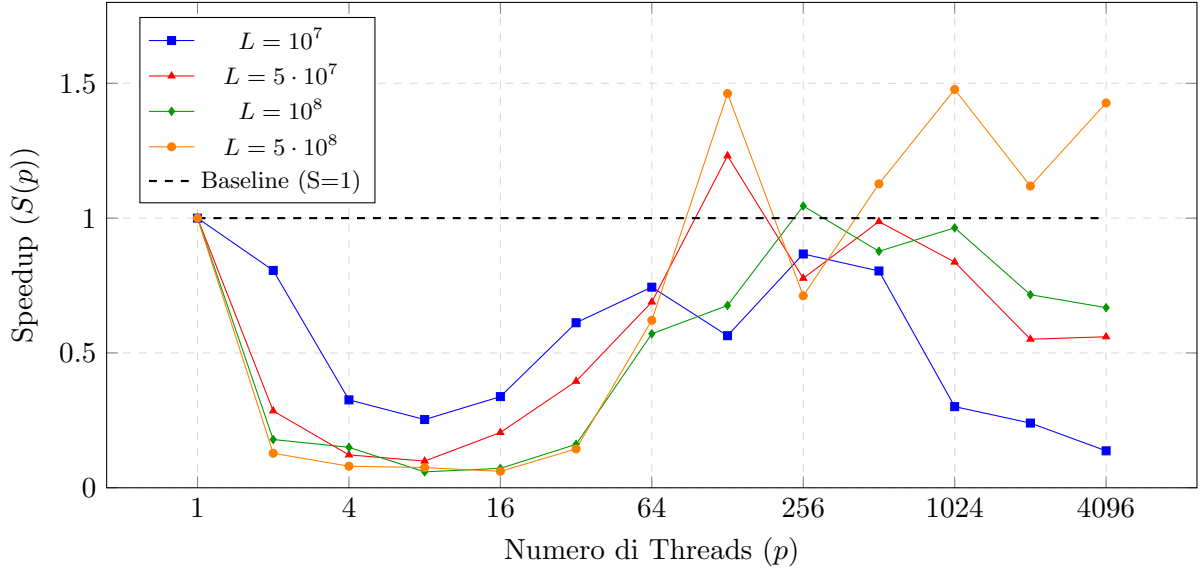


Figura 1: Rappresentazione dello speedup su un intervallo esteso. Il degrado prestazionale che si osserva quando si utilizzano più di 1024 thread viene analizzato nella Sezione 3.

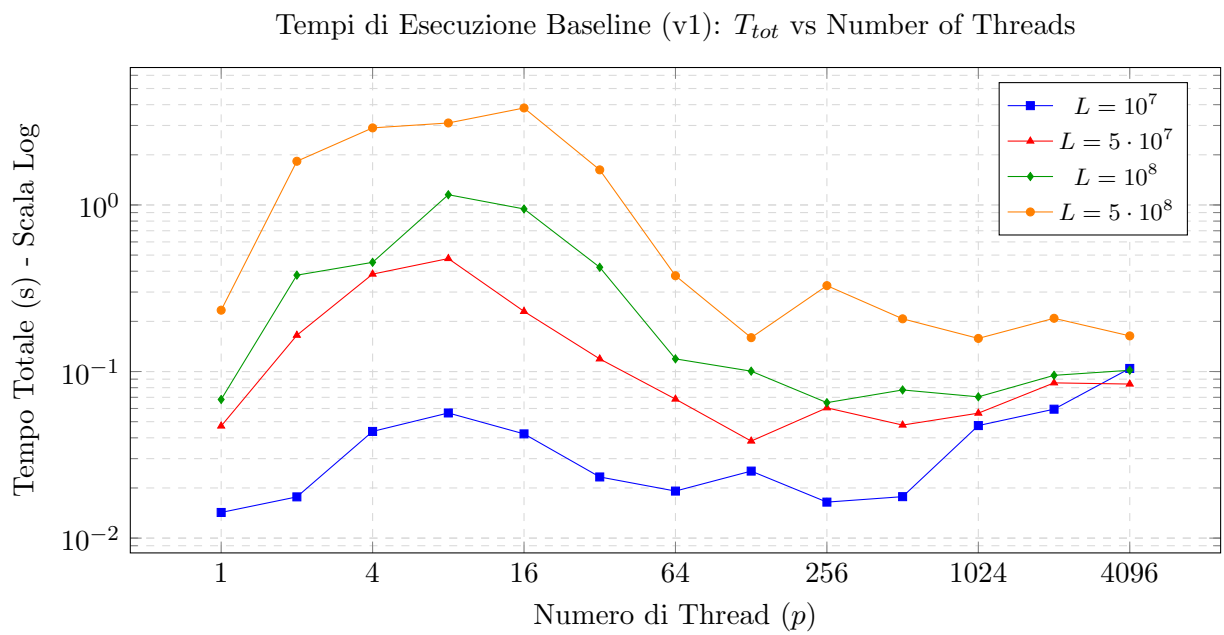


Figura 2: Andamento del tempo totale di esecuzione. Si nota chiaramente il picco di inefficienza nei primi stadi di parallelizzazione dovuto al false sharing, seguito da un miglioramento (non particolarmente significativo rispetto alla baseline) e infine una saturazione del guadagno prestazionale. Anche in questo caso, sono presenti più dettagli nella Sezione 3.

3 Analisi della non-linearità ed ipotesi sull'architettura

Analizzando i dati raccolti, emerge chiaramente che lo speedup non solo non ha un andamento lineare, ma per la maggior parte dei test è strettamente inferiore a 1. Paradossalmente, l'esecuzione parallela è significativamente più lenta rispetto a quella sequenziale. In base all'architettura del calcolatore, si sono ipotizzate tre cause principali.

3.1 False Sharing (Condivisione Falsa)

È il problema principale di questa implementazione. Nel codice, la matrice dei risultati è strutturata in modo che i contatori usati dai diversi thread per uno stesso intervallo (indicizzati con `local_histograms[bin][tid]`) si trovino posizionati uno accanto all'altro nella memoria fisica. Poiché il processore non legge la memoria un singolo dato alla volta, ma lavora "a blocchi" (le cosiddette linee di cache), i contatori di thread diversi finiscono inevitabilmente per condividere lo stesso blocco. Di conseguenza, ogni volta che un core aggiorna il contatore relativo al thread che sta eseguendo in quel momento, il protocollo di coerenza della cache considera "invalidato" l'intero blocco di memoria per tutti gli altri core. Questo comporta che i processori si ostacolano a vicenda, invalidandosi i dati e costringendosi a ricaricare continuamente le informazioni dalla memoria centrale, che è molto più lenta, azzerando di fatto i benefici della parallelizzazione.

3.2 Saturazione della Memory Bandwidth

Ogni thread esegue una semplice lettura, un controllo e una somma, ma richiede di scorrere un array di dimensione notevole. Aumentando il numero di thread, aumentano le richieste simultanee di lettura verso la memoria centrale. Tuttavia esiste un limite strutturale: la *Memory Bandwidth* (banda passante), ovvero la quantità massima di dati che il bus di sistema può trasferire dalla RAM ai processori nell'unità di tempo. Una volta saturata tale capacità, i processori sono costretti a mettersi in attesa che i dati arrivino per poter continuare a lavorare. In questo scenario, aggiungere ulteriori thread diventa inutile: la velocità complessiva del programma non dipende più dalla potenza di calcolo a nostra disposizione, ma da quanto velocemente la memoria riesce a fornire le informazioni su cui operare.

3.3 Oversubscription e Context Switch

La nostra macchina di test dispone di soli 8 core fisici, il che significa che già superando gli 8 thread chiediamo al sistema di gestire più processi di quante siano le unità calcolo reali (oversubscription). I dati evidenziano un fenomeno interessante: le prestazioni continuano a migliorare fino a circa 512 thread, per poi peggiorare nuovamente fino a 4096. Questo accade perché l'algoritmo utilizzato introduce frequenti cicli di stallo causati dalla latenza di accesso alla memoria principale. Avere un numero elevato di thread a disposizione aiuta il processore a evitare queste attese: appena un thread si blocca, la CPU lo mette in pausa e ne avvia un altro. Tuttavia, questo "trucco" ha un limite fisico. Nel caso limite di 4096 thread, il sistema operativo spende più tempo ad alternare i thread che a eseguire calcoli utili, provocando un nuovo peggioramento delle prestazioni.

4 Strategia di ottimizzazione e benchmark comparativo

Le ottimizzazioni successive mirano a correggere le inefficienze ipotizzate nella Sezione 3, modificando l'allocazione in memoria della matrice locale.

4.1 Prima Ottimizzazione: Trasposizione degli Indici (v2)

Per ridurre il *False Sharing* (Sez. 3.1), si è trasposta la matrice usando l'indicizzazione `local_histograms[tid][bin]`. In C, questo fa sì che ogni thread scriva su una riga di memoria dedicata e contigua di $7 \times 4 = 28$ byte.

I grafici (Figure 3 e 4) relativi al dataset da $5 \cdot 10^8$ elementi mostrano l'efficacia della modifica: il tempo di esecuzione con 8 thread crolla dai 3.28 secondi della versione base (v1) a 0.34 secondi (v2). Tuttavia, poiché 28 byte non riempiono un'intera *cache line* (64 byte), parte dei dati di thread diversi finisce ancora nello stesso blocco fisico, lasciando un residuo di *False Sharing*.

4.2 Seconda ottimizzazione: Cache Line Padding (v3)

Per risolvere definitivamente il problema, la riga dedicata a ciascun thread è stata "allungata" (*padding*) in modo da occupare esattamente un'intera *cache line*:

$$\text{Padding} = 16 \text{ int} \times 4 \text{ byte/int} = 64 \text{ byte} \quad (1)$$

I *bin* in eccesso (dal 7 al 15) rimangono inutilizzati e fungono da cuscinetto. In questo modo si garantisce che i core non condividano mai lo stesso blocco di memoria. Come si nota in Figura 3, questa ottimizzazione elimina il collo di bottiglia nei primi thread, portando a uno speedup corretto di 3.88x per 64 thread (0.058s).

4.3 Conferma del Memory Wall

Confrontando le tre versioni, si evince che lo *speedup* massimo della v3 si assesta poco sotto il 4x, senza crescere ulteriormente.

Questo dato conferma empiricamente quanto teorizzato in Sez. 3.2: per questo specifico algoritmo, il limite prestazionale non è la CPU, ma la *Memory Bandwidth*. Una volta saturata la capacità di trasferimento dati dalla memoria centrale, aggiungere altri thread diventa inutile e causa solo ritardi aggiuntivi a causa dell'*oversubscription* (Sez. 3.3).

Repository GitHub (Visibile solo nella Versione Docente):

<https://github.com/Wxlddd/challenge2algoritmi>

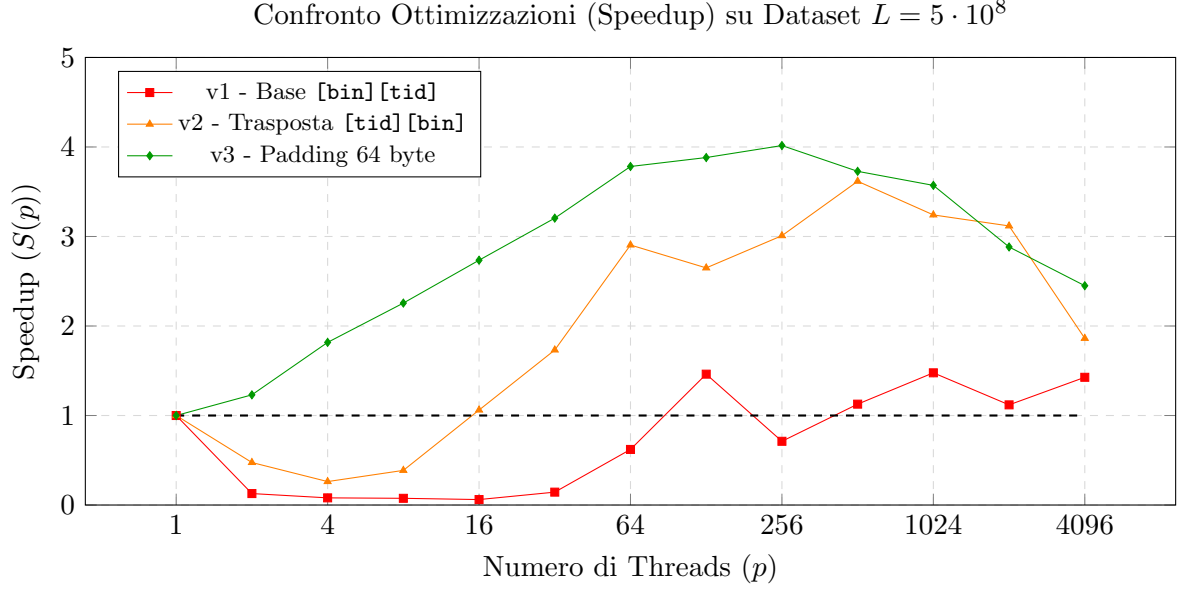


Figura 3: Confronto dello Speedup tra l'implementazione base e le due strategie di ottimizzazione della cache.

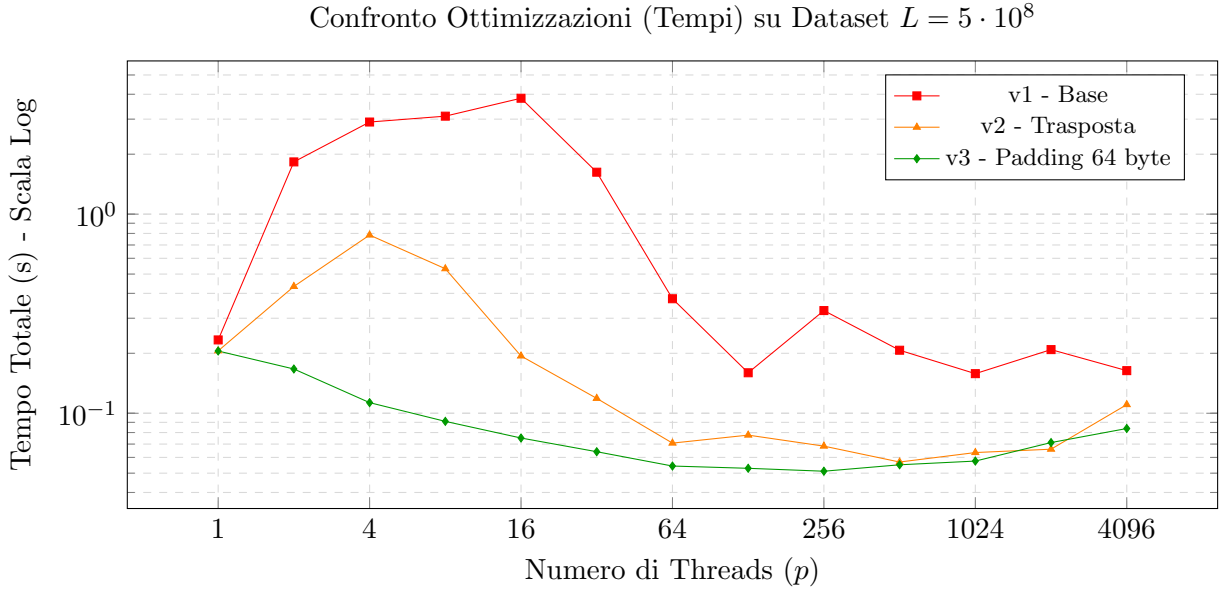


Figura 4: L'implementazione v3 annulla il ritardo nei primi cicli e restituisce l'andamento discendente atteso, fino allo stallo fisiologico per saturazione hardware.

5 Appendice: Tabelle dei Risultati per v2 e v3

Di seguito vengono riportati i dati completi raccolti durante i test prestazionali per le versioni ottimizzate v2 (trasposta) e v3 (padding a 64 byte). Si evidenzia che, ai fini dell'analisi comparativa, si preferisce fare riferimento alle rappresentazioni grafiche fornite nella Sezione 4, in quanto offrono una panoramica decisamente più immediata, chiara ed intuitiva dell'andamento prestazionale e della scalabilità al variare dei thread rispetto alla consultazione diretta dei dati numerici tabulati.

5.1 Dati Versione v2

Tabella 2: Tempi di esecuzione, speedup ed efficienza estesi fino a 4096 thread (v2 - Trasposta).

Lunghezza	Thread	T.tot (s)	T.par (s)	T.red (s)	Speedup	Efficienza
10 000 000	1	0.004614	0.004614	0.000000	1.000	1.0000
	2	0.009557	0.009557	0.000000	0.483	0.2414
	4	0.017265	0.017265	0.000000	0.267	0.0668
	8	0.011287	0.011286	0.000000	0.409	0.0511
	16	0.005319	0.005319	0.000000	0.867	0.0542
	32	0.003136	0.003135	0.000000	1.471	0.0460
	64	0.003468	0.003468	0.000000	1.330	0.0208
	128	0.003026	0.003026	0.000000	1.525	0.0119
	256	0.003782	0.003781	0.000001	1.220	0.0048
	512	0.005666	0.005666	0.000000	0.814	0.0016
	1024	0.011211	0.011211	0.000000	0.412	0.0004
	2048	0.020969	0.020969	0.000000	0.220	0.0001
	4096	0.045362	0.045361	0.000000	0.102	0.0000
50 000 000	1	0.021367	0.021366	0.000000	1.000	1.0000
	2	0.041574	0.041574	0.000000	0.514	0.2570
	4	0.149932	0.149932	0.000000	0.143	0.0356
	8	0.034210	0.034209	0.000000	0.625	0.0781
	16	0.021810	0.021810	0.000000	0.980	0.0612
	32	0.018839	0.018839	0.000000	1.134	0.0354
	64	0.012070	0.012070	0.000000	1.770	0.0277
	128	0.010031	0.010031	0.000000	2.130	0.0166
	256	0.010837	0.010837	0.000000	1.972	0.0077
	512	0.012856	0.012855	0.000000	1.662	0.0032
	1024	0.018301	0.018301	0.000000	1.168	0.0011
	2048	0.027870	0.027870	0.000000	0.767	0.0004
	4096	0.048493	0.048492	0.000000	0.441	0.0001
100 000 000	1	0.044362	0.044361	0.000000	1.000	1.0000
	2	0.131019	0.131019	0.000000	0.339	0.1693
	4	0.249057	0.249056	0.000000	0.178	0.0445
	8	0.093680	0.093679	0.000000	0.474	0.0592
	16	0.044912	0.044911	0.000000	0.988	0.0617
	32	0.033936	0.033936	0.000000	1.307	0.0409
	64	0.022770	0.022770	0.000000	1.948	0.0304
	128	0.015899	0.015899	0.000000	2.790	0.0218
	256	0.017411	0.017410	0.000000	2.548	0.0100
	512	0.020607	0.020607	0.000000	2.153	0.0042
	1024	0.021698	0.021698	0.000000	2.044	0.0020
	2048	0.032150	0.032150	0.000000	1.380	0.0007
	4096	0.058744	0.058743	0.000000	0.755	0.0002
500 000 000	1	0.205731	0.205731	0.000000	1.000	1.0000
	2	0.433256	0.433255	0.000001	0.475	0.2374
	4	0.785337	0.785337	0.000000	0.262	0.0655
	8	0.531943	0.531943	0.000000	0.387	0.0483
	16	0.193902	0.193901	0.000000	1.061	0.0663
	32	0.118751	0.118751	0.000000	1.732	0.0541
	64	0.070808	0.070807	0.000000	2.905	0.0454
	128	0.077661	0.077661	0.000000	2.649	0.0207
	256	0.068371	0.068371	0.000000	3.009	0.0118
	512	0.056880	0.056880	0.000000	3.617	0.0071

Continua nella pagina successiva

Tabella 2 – continua dalla pagina precedente

Lunghezza	Thread	T.tot (s)	T.par (s)	T.red (s)	Speedup	Efficienza
	1024	0.063475	0.063474	0.000000	3.241	0.0032
	2048	0.065985	0.065985	0.000000	3.118	0.0015
	4096	0.110486	0.110485	0.000000	1.862	0.0005

5.2 Dati Versione v3

Tabella 3: Tempi di esecuzione, speedup ed efficienza estesi fino a 4096 thread (v3 - Padding 64B).

Lunghezza	Thread	T.tot (s)	T.par (s)	T.red (s)	Speedup	Efficienza
10 000 000	1	0.004355	0.004354	0.000000	1.000	1.0000
	2	0.003297	0.003296	0.000000	1.321	0.6605
	4	0.002471	0.002471	0.000000	1.762	0.4405
	8	0.001917	0.001917	0.000000	2.271	0.2839
	16	0.006435	0.006435	0.000000	0.677	0.0423
	32	0.001456	0.001456	0.000000	2.990	0.0934
	64	0.002132	0.002132	0.000000	2.042	0.0319
	128	0.002899	0.002898	0.000000	1.502	0.0117
	256	0.004369	0.004369	0.000000	0.997	0.0039
	512	0.005897	0.005897	0.000000	0.738	0.0014
	1024	0.014759	0.014759	0.000001	0.295	0.0003
	2048	0.027253	0.027253	0.000000	0.160	0.0001
	4096	0.042993	0.042993	0.000000	0.101	0.0000
50 000 000	1	0.020511	0.020511	0.000000	1.000	1.0000
	2	0.029097	0.029096	0.000000	0.705	0.3525
	4	0.097653	0.097653	0.000000	0.210	0.0525
	8	0.013257	0.013257	0.000000	1.547	0.1934
	16	0.008309	0.008309	0.000000	2.469	0.1543
	32	0.005755	0.005755	0.000000	3.564	0.1114
	64	0.005947	0.005946	0.000000	3.449	0.0539
	128	0.008874	0.008874	0.000000	2.311	0.0181
	256	0.007723	0.007723	0.000000	2.656	0.0104
	512	0.011667	0.011667	0.000000	1.758	0.0034
	1024	0.011972	0.011971	0.000000	1.713	0.0017
	2048	0.023342	0.023342	0.000000	0.879	0.0004
	4096	0.040606	0.040605	0.000000	0.505	0.0001
100 000 000	1	0.044597	0.044597	0.000000	1.000	1.0000
	2	0.052620	0.052619	0.000000	0.848	0.4238
	4	0.025658	0.025658	0.000000	1.738	0.4345
	8	0.103575	0.103574	0.000000	0.431	0.0538
	16	0.014757	0.014757	0.000000	3.022	0.1889
	32	0.011971	0.011971	0.000000	3.725	0.1164
	64	0.020329	0.020328	0.000000	2.194	0.0343
	128	0.012678	0.012678	0.000000	3.518	0.0275
	256	0.017910	0.017910	0.000000	2.490	0.0097
	512	0.017398	0.017398	0.000000	2.563	0.0050
	1024	0.017740	0.017740	0.000000	2.514	0.0025
	2048	0.029486	0.029486	0.000000	1.512	0.0007
	4096	0.043828	0.043828	0.000000	1.018	0.0002

Continua nella pagina successiva

Tabella 3 – continua dalla pagina precedente

Lunghezza	Thread	T.tot (s)	T.par (s)	T.red (s)	Speedup	Efficienza
500 000 000	1	0.205383	0.205383	0.000000	1.000	1.0000
	2	0.166810	0.166809	0.000001	1.231	0.6156
	4	0.113049	0.113048	0.000000	1.817	0.4542
	8	0.091031	0.091031	0.000000	2.256	0.2820
	16	0.075094	0.075094	0.000000	2.735	0.1709
	32	0.064090	0.064090	0.000001	3.205	0.1001
	64	0.054305	0.054305	0.000000	3.782	0.0591
	128	0.052900	0.052900	0.000000	3.882	0.0303
	256	0.051147	0.051147	0.000000	4.016	0.0157
	512	0.055092	0.055092	0.000000	3.728	0.0073
	1024	0.057512	0.057512	0.000000	3.571	0.0035
	2048	0.071239	0.071238	0.000000	2.883	0.0014
	4096	0.083816	0.083816	0.000000	2.450	0.0006